

Informe Técnico

Proyecto: Aplicación de consola en JavaScript

Módulo 3 · Fundamentos de programación en JavaScript

1. Introducción

El presente informe documenta el desarrollo de una aplicación de consola en JavaScript, elaborada como evaluación del Módulo 3: Fundamentos de programación en JavaScript. El equipo de desarrollo de una plataforma de aprendizaje interactivo solicitó una aplicación que permitiera a los usuarios realizar operaciones básicas y trabajar con las estructuras fundamentales del lenguaje, como parte de una práctica aplicada de los conceptos vistos durante el módulo.

2. Objetivo del proyecto

Desarrollar una aplicación en consola que permita a los usuarios:

- Realizar operaciones matemáticas básicas.
- Implementar estructuras condicionales y de bucles.
- Usar funciones para modularizar el código.
- Trabajar con arreglos y objetos.

El código se estructuró siguiendo las mejores prácticas de desarrollo en JavaScript, priorizando la legibilidad, la modularización y el control de errores de entrada.

3. Descripción funcional

La aplicación se ejecuta íntegramente en la consola del navegador y se organiza en dos módulos accesibles desde un menú principal que se repite hasta que el usuario decide salir.

3.1 Menú principal

Implementado mediante un bucle while en la función `iniciarApp()`. En cada iteración se despliega un `prompt()` con tres opciones: operaciones matemáticas, gestión de estudiantes y salir. Una condicional `if / else if` procesa la opción elegida.

3.2 Módulo de operaciones matemáticas

La función `moduloOperaciones()` solicita dos números mediante `pedirNumero()` (que valida la entrada) y luego, a través de un `switch`, invoca al método correspondiente del objeto `calculadora`: sumar, restar, multiplicar, dividir o potencia. El resultado se muestra por `console.log()` y por `alert()`.

3.3 Módulo de gestión de estudiantes

Trabaja sobre el arreglo predefinido `estudiantes`, compuesto por objetos con las propiedades `nombre` y `nota`. Desde un submenú (`switch`) se puede:

- Ver todos los estudiantes (recorrido con `forEach`).
- Ver los estudiantes aprobados —nota mayor o igual a 6— filtrados con un bucle `for`.

- Calcular el promedio general de notas, recorriendo el arreglo con while.
- Agregar un nuevo estudiante, validando que el nombre no esté vacío y que la nota sea numérica.

4. Aspectos técnicos aplicados

4.1 Variables y tipos de datos

Se utilizaron let para valores que cambian durante la ejecución (por ejemplo, salir, entrada del usuario) y const para valores que no se reasignan (estudiantes, calculadora, funciones).

4.2 Estructuras condicionales

Se aplicó switch para los menús de opciones (operación matemática elegida, submenú de estudiantes) por tratarse de múltiples valores discretos, e if / else / while para las validaciones de entrada, donde la condición se evalúa de forma repetida hasta cumplirse.

4.3 Estructuras de repetición

Se usó while en el menú principal y en el cálculo del promedio (calcularPromedio), for en el filtro de estudiantes aprobados (obtenerAprobados) y el método de array forEach para recorrer y mostrar la lista de estudiantes, alternando así las tres formas de iteración vistas en el módulo.

4.4 Funciones y modularización

Cada responsabilidad del programa vive en su propia función: validación (validarNumero, pedirNumero), operaciones matemáticas (dentro del objeto calculadora), gestión de estudiantes (obtenerAprobados, calcularPromedio, mostrarEstudiantes, agregarEstudiante) y orquestación (moduloOperaciones, moduloEstudiantes, iniciarApp). Las funciones de orquestación llaman a las funciones más pequeñas, evitando código duplicado.

4.5 Arreglos y objetos

El arreglo estudiantes almacena objetos con propiedades nombre y nota, manipulado con push(), forEach() y recorridos manuales. El objeto calculadora agrupa los métodos matemáticos como propiedades, demostrando el uso de objetos para organizar comportamiento relacionado.

4.6 Validaciones

La función validarNumero() controla que el valor ingresado no esté vacío y sea numérico (isNaN). pedirNumero() reintenta la solicitud hasta recibir un valor válido. Al agregar un estudiante se valida además que el nombre no quede vacío. La división por cero se controla dentro del método dividir del objeto calculadora.

5. Estructura del código

El archivo script.js se organiza en las siguientes secciones, en este orden:

Sección	Contenido
Variables predefinidas	Arreglo de objetos estudiantes y objeto calculadora con los métodos matemáticos.
Validaciones	validarNumero() y pedirNumero(): evitan valores vacíos o no numéricos antes de operar.
Operaciones matemáticas	moduloOperaciones(): usa switch para elegir la operación y llama a los métodos de calculadora.

Gestión de estudiantes	obtenerAprobados() (for), calcularPromedio() (while), mostrarEstudiantes() (forEach), agregarEstudiante().
Menú principal	iniciarApp(): bucle while que muestra el menú hasta que el usuario elige salir.

6. Validación de requisitos de la consigna

A continuación se contrasta el desarrollo contra los aspectos técnicos, estructurales y funcionales solicitados en la consigna del módulo:

Aspecto evaluado	Cumple	Evidencia en el código
Aplicación correcta de fundamentos de JS	Sí	Uso de let/const, operadores, funciones flecha y tradicionales.
Uso de funciones para modularizar	Sí	calculadora, moduloOperaciones, moduloEstudiantes, iniciarApp, funciones de validación.
Estructuras de control correctas	Sí	switch en ambos módulos, if/else en validaciones, while en el menú principal.
Manejo de arreglos y objetos	Sí	Arreglo estudiantes de objetos {nombre, nota}; objeto calculadora con métodos.
Legibilidad y organización	Sí	Código separado por bloques comentados: variables, validaciones, operaciones, estudiantes, menú.
Buenas prácticas de nombres	Sí	Nombres descriptivos en español acorde al dominio (pedirNumero, obtenerAprobados, etc.).
Comentarios explicativos	Sí	Bloques de comentarios por sección y comentarios en línea en los puntos clave.
Ejecución sin errores	Sí	Validado con node --check script.js antes de la entrega.
Validación de entradas	Sí	Rechaza campos vacíos, valores no numéricos y controla la división por cero.

7. Pruebas realizadas

- Verificación de sintaxis con node --check script.js, sin errores.
- Ejecución manual del menú principal probando las tres opciones (operaciones, estudiantes, salir).
- Prueba de validaciones ingresando valores vacíos, texto no numérico y división por cero.
- Prueba de alta de estudiantes y verificación del recálculo del promedio y del listado de aprobados.

8. Reflexión y conclusiones

El desarrollo permitió aplicar de forma integrada los contenidos de las cinco lecciones del módulo: sintaxis básica, variables y condicionales, arreglos y ciclos, funciones, y objetos. Separar el programa en funciones

pequeñas con una única responsabilidad facilitó tanto la lectura del código como la incorporación de validaciones sin duplicar lógica.

El mayor desafío estuvo en decidir qué estructura de repetición usar en cada caso (for vs. while vs. forEach) buscando que cada elección fuera la más clara para su propósito, en lugar de usar siempre la misma. Como mejora futura, se plantea persistir la lista de estudiantes en localStorage y reemplazar los prompt() por una interfaz HTML con formularios.